BuildIT

PC Components Online Store — Mini-Capstone Project

Code Reviewer — What Every Script and Variable Is For

Companion to the Mini-Capstone Reviewer

A plain-English map of the source code:
which scripts run, what each variable holds, and where to find it.

Covers: `app.js` · `conn.js` · `admin.js` · `nav.js`
`create-admin.js` · `make-admin.js` · EJS views · inline scripts

Last updated: October 15, 2026 — reflects the 12-part category update, the rewritten make-admin.js, and local product photos in public/images/.

What's inside this code reviewer

1. The scripts — what runs, where it runs, and how to start it
2. `app.js` map — where everything lives, top to bottom
3. Configuration variables (the `.env` file and server setup)
4. Server constants — the settings that never change
5. Session variables — the server's memory of each visitor
6. Request variables — what each form and page sends
7. Key data objects — `form`, `payment`, `stats`, `flash`
8. Variables passed to each EJS view
9. Database columns — the variables that live in MySQL
10. Front-end script variables (`admin.js`, navbar, checkout, OTP, builder)
11. Naming cheat sheet + quick self-test

1. The scripts — what runs, where it runs

Script	Runs in	How it starts	What it is for
`app.js`	Server	`npm start` (nodemon) or `node app.js`	**The whole backend.** Creates the database tables, defines every route, handles login/session/CSRF security, checkout validation, OTP verification, order transactions, and the admin API. When someone says "the app", this is the file.
`conn.js`	Server	Automatically, whenever a script does `require('./conn')`	Creates the MySQL **connection pool** (up to 10 parallel connections) using the DB values from `.env`, and exports it as `db`. Used by `app.js`, `create-admin.js`, and `make-admin.js`.
`create-admin.js`	Terminal (one-shot)	`node create-admin.js`	Interactive tool: asks for full name, email, and a hidden password, hashes it with bcrypt (cost 12), and inserts it into the `admins` table. **This is the official way to make an admin.**
`make-admin.js`	Terminal (one-shot)	`node make-admin.js <email>`	Promotes an **existing customer** to an admin: looks the email up in `users`, then copies the account (name + the same bcrypt hash) into the `admins` table — where admin login actually checks. The person keeps their current password. Prints a clear message if the email isn't registered, or does nothing if they're already an admin.
`admin.js` (in `public/`)	Browser	Loaded by `admin.ejs`: `<script src="/admin.js">`	All admin-page interactivity: mobile sidebar, the add/edit product dialog, the discount calculator, live product search, and confirm-before-deleting.

`admin.js` (in project root)	—	Never runs directly	Source/reference copy of the same file. **Always copy it into `public/` after editing** — the browser only ever loads the `public/` copy.
`nav.js`	—	Never loaded directly	Source/reference copy of the navbar script (desktop dropdowns, mobile hamburger, Esc to close). The identical code is **inlined at the bottom of `index.ejs`**, which is what actually runs.
`admin-table.sql`	MySQL	`mysql -u root -p buildit < admin-table.sql`	Optional backup way to create the `admins` table by hand, with commented instructions for inserting a hashed admin. The app creates this table automatically anyway.
`package.json` scripts	Terminal	`npm start` / `npm test`	`start` runs `nodemon app.js` (auto-restarts on file changes). `test` is the default placeholder that just prints an error — there are no automated tests.
Inline `<script>` blocks in the EJS views	Browser	Part of the page	`index.ejs` → navbar menus · `checkout.ejs` → payment tabs + live shipping math · `otp.ejs` → countdown timer + digits-only input · `pcbuilder.ejs` → build total calculator.

> **Rule of thumb:** code in `app.js` / `conn.js` runs on the *server* (sees the database and secrets). Code in `public/*.js` and the EJS inline blocks runs in the *browser* (sees only the HTML it is attached to). EJS templates run on the server but produce the browser's HTML.

2. `app.js` map — where everything lives, top to bottom

`app.js` is one long file (~1,400 lines). Here is its layout, in order:

#	Section	What's in it
1	**Imports & app setup**	Requires `express`, `crypto`, `bcrypt`, `express-session`, `path`, `dotenv`. Creates the Express app, refuses to start without `SESSION_SECRET`, sets EJS as the view engine, adds body-parsing and static-file middleware, configures the session cookie (24 hours).
2	**Category constants**	`CATEGORIES` — the 12 parts list with icons/badges; `LEGACY_CATEGORY_NAMES` — the rename map for old databases.
3	`initDB()`	Creates/upgrades all 7 tables, migrates legacy category names, adds missing categories, and seeds demo products on empty databases. Runs once at startup.
4	**Middleware**	`requireLogin()` — blocks customer pages when not logged in.
5	**Store routes**	`/` (catalog + filter/sort), `/login`, `/register`, `/cart` (+ add/update/remove), `/logout`.
6	**Checkout config**	`PH_REGIONS`, `PAYMENT_METHODS`, `EWALLETS`, `SHIPPING`, and helpers `shippingFor()`, `orderNumber()`, `luhnValid()`, `cardBrand()`.
7	**Checkout helpers**	`loadCart()`, `renderCheckout()`, `validateCheckout()` (cleans and checks every field).
8	**OTP block**	OTP constants, `generateOtp()`, `hashOtp()`, `otpMatches()`, `otpIsActive()`, `maskNumber()`, `issueOtp()`.
9	**Checkout & order routes**	`GET/POST /checkout`, `GET/POST /checkout/otp`, `/checkout/otp/resend`, the transactional `placeOrder()`, `GET /orders/:id`.
10	**PC builder**	`BUILDER_SLOTS` (the 12 dropdown definitions), `GET /pcbuild`, `POST /pcbuild/add-all`.
11	**Admin block**	`requireAdmin()`, `csrfProtect()`, `setFlash()`, `parseProductForm()`, `describeDbError()`, `dbAction()`, the `adminRouter` routes, admin login/logout, `DUMMY_HASH`.
12	**Boot**	`initDB().then(() => app.listen(PORT))` — the server only starts after the database is ready.

3. Configuration variables (the `.env` file and server setup)

Variable	Where it lives	What it does
`PORT`	`.env` (fallback `3000`)	The port the server listens on: `http://localhost:3000`.
`SESSION_SECRET`	`.env`	Secret key that signs the session cookie so it can't be forged. `app.js` **throws and refuses to boot** if it's missing.

`DB_HOST`	`.env` (fallback `localhost`)	MySQL server address.
`DB_USER` / `DB_PASSWORD`	`.env` (fallback `root` / empty)	Login credentials for MySQL.
`DB_NAME`	`.env` (fallback `buildit`)	The database the app uses.
`connectionLimit` = `10`	`conn.js`	How many simultaneous MySQL connections the pool may open.
`cookie.maxAge` = `86400000`	`app.js` session config	Milliseconds a login stays valid: 24 hours.
`bcrypt` cost = `12`	register, admin login, `create-admin.js`	Hashing strength (2^{12} rounds). Higher = slower but safer.
`views` folder & EJS engine	`app.js`	Tells Express to render the `views/*.ejs` templates.
`DUMMY_HASH`	`app.js` admin login	A real bcrypt hash of a fake password, always compared at login so response times can't reveal which admin emails exist.

> **Never commit `.env`.** It holds the database password and the session secret; it is listed in `.gitignore` for that reason.

4. Server constants — the settings that never change

Checkout & payment

Constant	Value	Purpose
`PH_REGIONS`	17 pairs like `['NCR', 'NCR - National Capital Region…']`	Dropdown options for the address form; also used to pretty-print the region on the receipt.
`REGION_CODES`	Set built from PH_REGIONS	Fast lookup to reject made-up regions.
`PAYMENT_METHODS`	`{ debit:'Debit Card', ewallet:'E-Wallet', cod:'Cash on Delivery' }`	Valid payment modes + their display labels. The OTP list is derived from its keys.
`EWALLETS`	`['GCash','Maya','GrabPay','ShopeePay']`	Allowed e-wallet providers.
`SHIPPING`	`{ ncr: 150, other: 250, freeAbove: 30000 }`	Shipping fees in ₱: Metro Manila vs. rest of PH; free at ₱30,000+.

OTP verification

Constant	Value	Purpose
`OTP_METHODS`	Set of every payment method	Means *all* payment modes require a code before an order is created.
`OTP_LENGTH`	`6`	Code length in digits.
`OTP_TTL_MS`	`300000` (5 min)	How long a code stays valid.
`OTP_MAX_ATTEMPTS`	`5`	Wrong guesses before the code (and checkout) is burned.
`OTP_RESEND_COOLDOWN_MS`	`30000` (30 s)	Minimum wait between "send a new code" requests.

The 12-part catalog & builder

Constant	Value	Purpose
`CATEGORIES`	12 objects: Case, Motherboard, CPU, RAM, Storage (SSD/HDD), GPU, PSU, Cooling System, Monitor, Keyboard, Mouse, Power Cable — each with an icon, badge, and label	The single source of truth for the store's category list. `initDB()` guarantees all 12 exist.
`LEGACY_CATEGORY_NAMES`	`Processors→CPU`, `Graphics Cards→GPU`, `Motherboards→Motherboard`, `RAM Memory→RAM`	Startup migration so databases made with the old

		names are upgraded instead of broken.
`BUILDER_SLOTS`	12 objects: `{ field, category, label, icon }` — e.g. `{ field:'psuId', category:'PSU', … }`	Defines the PC Builder: which form field name, which category, which caption and icon each dropdown uses. One list drives the page, the query, and the add-to-cart handler.

Validation limits (in `parseProductForm` and helpers)

Limit	Value	Why
`MAX_PRICE`	`99999999.99`	Biggest number MySQL `DECIMAL(10,2)` can hold.
Product name / badge / image / description	255 / 50 / 255 / 2000 chars	Matches the database column sizes.
Stock	Whole number, 0 … 2,147,483,647	Must fit an `INT`.
Phone pattern	`^(09\d{9}|\+639\d{9})$`	Philippine mobile numbers only; stored normalized to `09…`.
Card number	16 digits + Luhn checksum; expiry MM/YY not past; CVV 3–4 digits	Catches typos; details are then discarded, never stored.

5. Session variables — the server's memory of each visitor

Everything on `req.session` lives on the **server** (the browser only holds an opaque cookie). It disappears on logout or after 24 hours.

Variable	Set when	Meaning
`req.session.userId`	Customer logs in	Id of the logged-in customer — the key for every `requireLogin` check.
`req.session.userName`	Customer logs in	The customer's full name, used in navbars and page titles.
`req.session.adminId`	Admin logs in (after `regenerate()`)	Id of the admin — re-verified against the `admins` table on every admin request.
`req.session.csrfToken`	First page that runs `csrfProtect`	Random 32-byte secret. Copied to `res.locals.csrfToken` so forms can embed it; every protected POST must send it back as `_csrf`.
`req.session.flash`	`setFlash(type, text)`	One-shot `{ type: 'success'|'error', text }` message shown after a redirect, then deleted.
`req.session.pendingCheckout`	POST /checkout needs OTP	The half-finished order (see below) — deleted on success or after too many wrong codes.

The `pendingCheckout` object

Field	Holds
`form`	The validated checkout form (address + phone + payment selection).
`payment`	`{ method, provider, reference, status }` — the payment decision (see section 7).
`createdAt`	When checkout started (`Date.now()`).
`otpHash`	SHA-256 hash of the code — the real code is never stored.
`otpDemo`	The plain code, kept only because there's no SMS gateway — it's shown on the OTP page for the demo.
`otpIssuedAt` / `otpExpiresAt`	Timestamps for the 5-minute TTL and the 30-second resend cooldown.
`otpAttempts`	Wrong guesses so far (max 5).

6. Request variables — what each form and page sends

After a POST, the parsed body is `req.body`. Query-string values are `req.query`; route-path values are `req.params`.

Endpoint	Variables received
`POST /login`	`email`, `password`
`POST /register`	`name`, `email`, `password`, `confirmPassword`, `terms`
`GET /`	`?category=` (only accepted if it exists in the DB), `?sort=` (`low` | `high`)
`POST /cart/add`	`productId`
`POST /cart/update`	`cartItemId`, `action` (`increase` | `decrease`)
`POST /cart/remove`	`cartItemId`
`POST /pcbuild/add-all`	12 slot fields: `caseId`, `moboId`, `cpuId`, `ramId`, `storageId`, `gpuId`, `psuId`, `coolingId`, `monitorId`, `keyboardId`, `mouseId`, `powerCableId` — blanks are ignored, ids are re-validated against the DB
`POST /checkout`	`fullName`, `phone`, `street`, `barangay`, `city`, `province`, `region`, `postalCode`, `paymentMethod`, `ewalletProvider`, `ewalletNumber`, `cardName`, `cardNumber`, `cardExpiry`, `cardCvv`, `_csrf` (card fields only for debit)
`POST /checkout/otp`	`otp` (6 digits), `_csrf`
`POST /admin/products` (+ `/:id`)	`name`, `category`, `price`, `oldPrice`, `stock`, `badge`, `description`, `image`, `_csrf`
`POST /admin/login`	`email`, `password`, `_csrf`
`GET /orders/:id`	`req.params.id` — must be an integer *and* belong to the logged-in user

7. Key data objects — `form`, `payment`, `stats`, `flash`

Object	Shape	Where it's used
`payment`	`method`: 'debit'|'ewallet'|'cod' · `provider`: 'Visa'/'Mastercard'/'GCash'... or null · `reference`: e.g. "Visa debit ending 4242" · `status`: 'Paid' or 'Pay on delivery'	Built by `validateCheckout()`, stored in `orders` (the real card number is thrown away).
`form` (checkout)	Trimmed strings: `fullName, phone, street, barangay, city, province, region,`	Cleaned by `validateCheckout()`; returned with any error so the page can

	`postalCode, paymentMethod, ewalletProvider, ewalletNumber, cardName`	re-fill what the user typed. Stored in `pendingCheckout` for the OTP step.
`stats`	`products` · `customers` · `lowStock` (stock ≤ 5) · `cartValue` (all carts at current prices)	The four numbers on the admin dashboard cards.
`flash`	`{ type: 'success'|'error', text: 'Saved "…" to the database.' }`	Set by `setFlash()` after admin actions; consumed once by the dashboard, then deleted.
`hero`	`{ title: 'LEVEL UP YOUR GEAR', subtitle, ctaText, ctaLink }`	Home-page banner content, defined in the `/` route.
Return values of helpers	`validateCheckout()` → `{ error, form }` or `{ form, payment }` · `parseProductForm()` → `{ error }` or `{ data }`	The "either it failed with a message, or here's the safe data" pattern used all over the app.

8. Variables passed to each EJS view

Everything in `res.render('view', { … })` becomes a variable you can use inside that template with `<%= name %>`.

View	Variables it receives
`index.ejs`	`hero`, `categories` (each with a live `productCount`), `products`, `selectedCategory`, `sort`, `user`
`login.ejs` / `register.ejs` / `admin-login.ejs`	`error` (or null)
`cart.ejs`	`cart` (rows with line `subtotal`), `total`, `userName`
`checkout.ejs`	`cart`, `subtotal`, `regions`, `ewallets`, `shippingRates`, `error`, `form` (refill), `userName`
`otp.ejs`	`error`, `cart`, `subtotal`, `shippingFee`, `total`, `demoCode`, `sentTo` (masked number), `methodLabel`, `reference`, `expiresAt`, `userName`
`order-success.ejs`	`order` (the full row), `items`, `regionName`, `orderNo` (BIT-000042), `paymentLabel`, `userName`
`pcbuilder.ejs`	`components` — the 12 `BUILDER_SLOTS`, each with its `items` (in-stock products); `user`
`admin.ejs`	`admin` ({name, email}), `flash`, `stats`, `products`, `categories` (drives the dropdown), `customers`

> **`res.locals.csrfToken`** is not passed manually — `csrfProtect` attaches it to every protected view automatically, so any form can write `<input type="hidden" name="_csrf" value="<%= csrfToken %>">`.

9. Database columns — the variables that live in MySQL

These are the "variables" the server reads with `SELECT` and writes with `INSERT/UPDATE`.

Table	Columns (type)
`users`	`id` int PK · `full_name` · `email` UNIQUE · `password` (bcrypt hash) · `created_at`
`admins`	`id` PK · `full_name` · `email` UNIQUE · `password` · `created_at`
`categories`	`id` PK · `name` (the 12 parts) · `icon` (bi-...) · `badge` (HOT/SALE...) · `itemCount` (label)
`products`	`id` PK · `name` · `category` (matches a categories.name) · `price` DECIMAL(10,2) · `oldPrice` (discount anchor, camelCase because it's JS-style) · `stock` int · `image` (external `http(s)://` URL *or* a `/images/…` path pointing at a file in `public/images/`) · `badge` · `description` TEXT
`cart_items`	`id` PK · `user_id` FK→users (cascade) · `product_id` FK→products (cascade) · `quantity` · UNIQUE(user_id, product_id)
`orders`	`id` PK · `user_id` FK · address: `full_name, phone, street, barangay, city, province, region, postal_code` · payment: `payment_method, payment_provider, payment_reference, payment_status` · money: `subtotal, shipping_fee, total` · `status` (default 'Processing') · `created_at`
`order_items`	`id` PK · `order_id` FK (cascade) · `product_id` FK (set null) · snapshot copies of `name`, `price` · `quantity`

10. Front-end script variables

`public/admin.js` — the admin page

Variable	What it is
`$`	Shortcut for `document.querySelector(sel)` — used everywhere for brevity.
`sidebar`, `scrim`, `menuToggle`	The three pieces of the mobile slide-out menu; `setMenu(open)` toggles them together.
`oldEl` (#pOldPrice), `discEl` (#pDiscount), `priceEl` (#pPrice), `note` (#discountNote)	The discount calculator: type an old price + % → new price is filled in (and vice versa); `note` shows friendly hints like "10% off. Customers save ₱…".
`peso(n)`, `num(el)`	Helpers: format a number as ₱ with commas; parse an input to a number or `null` if invalid.
`dialog`, `form`, `title`	The add/edit popup. `openDialog('add')` or `openDialog('edit', data)` resets the form, points `form.action` to the right URL, and fills every field.
`btn.dataset.*`	Each row's Edit button carries the product's data in HTML attributes (`data-id`, `data-price`…) — that's how the browser knows the product without calling the server.
`search`, `rows`, `empty`	Live product search: hides table rows that don't contain the typed text and shows an "empty" message when nothing matches.

Navbar script (inlined in `index.ejs`; source copy in `nav.js`)

Variable	What it is
`dropdowns`	All desktop category menus (NodeList of `.nav-dropdown`).
`mobileToggle`, `mobileMenu`	The hamburger button and the menu it opens below 768px.
`navbar`	Used to decide if a click was "outside" the menu.
`closeDropdowns(except)`, `setMobile(open)`	The two state-changers; Esc and outside-clicks call them.

`checkout.ejs` inline script

Variable	What it is
`rates`	Copy of `SHIPPING` injected by EJS (`ncr`, `other`, `freeAbove`) so the browser can preview shipping fees without a server call.
`subtotal`	Read from the summary card's `data-value` attribute.
`syncPayment()`	

	Shows only the chosen payment panel and changes the button label ("PAY WITH DEBIT CARD"…).
`syncShipping()`	Recomputes the shipping fee + total as the region changes — the *same* rule as the server (free above ₱30k, else NCR ₱150 / other ₱250). The server still recalculates before charging.

`otp.ejs` inline script

- `expires` — timestamp from `data-expires`; `tick()` shows a live "Code expires in 04:59" countdown (the server enforces the real expiry).
- `input` — forced to digits only, max 6 characters.
- Submit button is disabled after the first click to prevent double orders.

`pcbuilder.ejs` inline script

- `calculateTotal()` — adds up the `data-price` of the selected option in every `.component-select` dropdown and writes it to `#buildTotal`.

11. Naming cheat sheet + quick self-test

Where a variable lives — the four "namespaces"

Write this	Means	Example
`req.body.x`	Value posted by a form	`req.body.email`
`req.query.x`	Value in the URL after `?`	`req.query.sort`
`req.params.x`	Value in the URL path	`req.params.id` in `/orders/42`
`req.session.x`	Server-side memory for this visitor	`req.session.userId`
`res.locals.x`	Value handed to an EJS view automatically	`res.locals.csrfToken`

Naming conventions used in this project

- **camelCase** everywhere in JavaScript (`shippingFee`, `pendingCheckout`).
- **snake_case** for most database columns (`full_name`, `created_at`, `payment_method`) — except `products.oldPrice`, which is camelCase to match the JS code.
- `$name` in front-end scripts = "result of querySelector".
- `UPPER_CASE` = a constant that never changes (`SHIPPING`, `OTP_LENGTH`).
- `el` / `El` suffix = an HTML element (`regionEl`, `oldEl`).
- `?` inside SQL = a placeholder safely filled by `db.execute(sql, params)`.
- `data-*` attributes = how server-rendered HTML hands data to browser scripts.
- `*El` / `view` locals = the keys you see in `res.render(...)`.

Quick self-test (answers below)

1. Where is the 5-minute OTP lifetime defined?
2. Which variable tells the server who is logged in?
3. What is `DUMMY_HASH` for?
4. Which list decides how many dropdowns the PC Builder has?
5. Your boss types a discount into the admin form — which script computes the new price?
6. Why does `validateCheckout()` exist?
7. What happens to `pendingCheckout` after a correct OTP?
8. You edit root `admin.js` but nothing changes in the browser. Why?

> **Answers:** (1) `OTP_TTL_MS` in app.js. (2) `req.session.userId` (customers) / `req.session.adminId` (admins). (3) A real bcrypt hash of a fake password, compared on every admin login so timing can't reveal valid emails. (4) `BUILDER_SLOTS`. (5) `public/admin.js` — the discount helpers `refreshNote()` / `discountFromPrices()`. (6) To clean and check every field once, server-side, and return either `{error, form}` or `{form, payment}`.

> (7) It is deleted — the order is placed immediately after. (8) The browser loads `public/admin.js`; copy your changes there (root `admin.js` is only the source copy).

Generated from the source code in `/Mini-Capstone` after the 12-part category update. Companion document: *BuildIT Mini-Capstone Reviewer* (project overview, features, security, and defense Q&A).